

Paso 1. Diagnóstico de la tabla cruda

Identifica atributos no atomicos.

productos_codigo, productos_nombre, cliente_direccion, vendedor_nombre, categorias, descuentos, metodos_pago, cantidades, precios.

Marca datos repetidos por cliente, vendedor, producto, categoria y pago.

Datos repetidos por Cliente:

maria gomez: doc(cc101), direccion(calle 10 #5-20), ciudad(Bogota), correo(maria.gomez@mail.com), telefono(3101112233).

Por vendedor:

id_vendedor(VEN01) vendedor_nombre(Ana torres), vendedor_zona(norte)

Por producto:

producto_nombre(teclado mecanico) producto_categoria(perifericos),
producto_codigo(P002).

Por pago:

metodo_pago(tansferencia), entidad_pago(bancolombia)

Explica al menos tres anomalias: insercion, actualizacion y eliminacion.

Anomalía de inserción: no se puede registrar un nuevo cliente en el sistema si este aún no ha realizado ninguna compra.

Anomalía de actualización: si un cliente cambia alguno de sus datos numero de telefono por ejemplo, tocaria buscar todas las veces que ha aparecido en el historial y actualizarla una por una y no se actualiza siquiera una línea la base de datos quedaría con inconsistencias.

Anomalía de eliminación: si decidimos cancelar y borrar la compra de laura rojas también eliminamos todos sus datos personales.

Redacta una lista inicial de dependencias funcionales.

Cliente_doc — cliente_nombre, cliente_email, cliente_telefono, cliente_direccion, cliente_ciudad (El documento de identidad determina de forma única los datos del cliente).

vendedor_id — vendedor_nombre, vendedor_zona (El ID del vendedor determina de forma única su nombre y su zona).

producto_codigo — producto_nombre, categoria (El código del producto determina de forma única su nombre y su categoría).

venta_id — fecha_venta, cliente_doc, vendedor_id, metodo_pago, entidad_pago (El ID de la venta determina la fecha de compra, el cliente que compró, el vendedor que atendió y el método de pago usado).

Paso 2. Aplicación de 1FN

Transforma los campos multivaluados en registros separados.

Nombre de la Columna	Registro 1	Registro 2	Registro 3	Registro 4	
venta_id	V1001	V1001	V1002	V1002	
fecha_venta	2026-04-01	2026-04-01	2026-04-02	2026-04-02	
cliente_doc	CC101	CC101	CC102	CC102	
cliente_nombre	Maria Gomez	Maria Gomez	Juan Perez	Juan Perez	
cliente_email	maria.gomez@mail.com	maria.gomez@mail.com	juan.perez@mail.com	juan.perez@mail.com	
cliente_telefono	3101112233	3101112233	3155558899	3155558899	
cliente_direccion	Calle 10 #5-20	Calle 10 #5-20	Carrera 8 #20-15	Carrera 8 #20-15	

cliente_ciudad	Bogota	Bogota	Bogota	Bogota	
vendedor_id	VEN01	VEN01	VEN02	VEN02	
vendedor_nombre	Ana Torres	Ana Torres	Carlos Ruiz	Carlos Ruiz	
vendedor_zona	Norte	Norte	Centro	Centro	
producto_codigo	P001	P002	P003	P001	
producto_nombre	Mouse USB	Teclado mecanico	Monitor 24	Mouse USB	
categoria	Perifericos	Perifericos	Pantallas	Perifericos	
cantidad	2	1	1	1	
precio_unitario	45000	180000	720000	45000	
descuento	0	0	20000	0	
metodo_pago	Transferencia	Transferencia	Tarjeta credito	Tarjeta credito	

entidad_pago	Bancolombia	Bancolombia	Visa	Visa	
total_venta	270000	270000	745000	745000	

Cada fila debe representar una venta con un solo producto asociado.

Venta V1001 (Cliente Maria Gomez): Contenía originalmente dos códigos de productos en la misma celda (P001, P002). Se desglosa en **dos filas individuales**: la primera fila asocia la venta V1001 exclusivamente con el producto P001 (Cantidad: 2, Precio: 45000) , y la segunda fila asocia la misma venta V1001 con el producto P002 (Cantidad: 1, Precio: 180000).

Venta V1002 (Cliente Juan Perez): Contenía originalmente dos productos (P003, P001). Se divide en **dos filas individuales**: la primera fila vincula la venta al Monitor P003 aplicando un descuento de 20000 , y la segunda fila la vincula al Mouse P001 con descuento 0.

Venta V1003 (Cliente Maria Gomez): Presentaba una lista de tres productos (P004, P002, P005). Se expande en **tres filas individuales** en la tabla final, donde cada registro almacena de forma unívoca un solo producto con su respectiva cantidad y precio asignado por posición dentro de la venta.

Venta V1004 (Cliente Laura Rojas): Contenía dos productos (P003, P006). Se separa en **dos filas individuales**: una fila para el Monitor P003 (Cantidad: 2) y otra para la Webcam P006 (Cantidad: 1, Descuento: 10000).

Verifica que no existan listas separadas por comas ni grupos repetitivos.

Bajo la estructura original, una sola fila pretendía actuar como una factura y un contenedor de múltiples artículos a la vez. Al aplicar la 1FN, se ha erradicado este patrón de diseño erróneo. No existen columnas duplicadas ocultas (como producto_1, producto_2) ni celdas con datos multidimensionales. La cardinalidad de los artículos comprados se refleja de manera vertical y uniforme a través del crecimiento de filas en la tabla, asegurando que cada intersección de columna y fila contenga exactamente un solo valor del dominio correspondiente.

Propone una clave compuesta provisional para la tabla en 1FN.

El campo venta_id funcionaba como el identificador único de cada registro. Sin embargo, al transformar la tabla a la Primera Forma Normal (1FN) y desglosar las listas de productos en filas independientes, el venta_id comienza a repetirse.

Clave Primaria Compuesta Provisional: (venta_id, producto_codigo)

La combinación de ambos atributos asegura que cada fila de la tabla en 1FN sea completamente única. Mientras que el venta_id se repite para agrupar los artículos de una misma compra, el producto_codigo identifica de forma unívoca a cada artículo dentro de ese grupo.

Paso 3. Aplicación de 2FN

Evalúa si la tabla en 1FN usa una clave compuesta.

La tabla en 1FN(si es compuesta) utiliza la clave primaria compuesta (venta_id, producto_codigo). Sí se requiere una clave compuesta porque el venta_id se repite por cada artículo vendido y el producto_codigo identifica de forma única a cada producto dentro de esa misma orden.

Separa los atributos que dependen solo de una parte de la clave.

Dependen solo de: **venta_id: fecha_venta, cliente_doc, cliente_nombre, cliente_email, cliente_telefono, cliente_direccion, cliente_ciudad, vendedor_id, vendedor_nombre, vendedor_zona, metodo_pago y entidad_pago.**

Identifica entidades como venta, cliente, vendedor, producto y detalle de venta.

Las entidades identificadas para el sistema son:

- cliente (Guarda los datos personales de los compradores)
- vendedor (Guarda la información de los asesores comerciales)
- producto (Guarda el catálogo de artículos disponibles)
- venta (Funciona como la cabecera o resumen de la factura)
- detalle_venta (Une cada venta con los productos específicos comprados)

Define claves primarias y foraneas para las entidades resultantes

Tabla: cliente

Clave Primaria (PK): cliente_doc

Claves Foráneas (FK): No tiene.

Tabla: vendedor

Clave Primaria (PK): vendedor_id

Claves Foráneas (FK): No tiene.

Tabla: producto

Clave Primaria (PK): producto_codigo

Claves Foráneas (FK): No tiene (*en este paso 2FN*).

Tabla: venta

Clave Primaria (PK): venta_id

Clave Foránea (FK1): cliente_doc (Apunta a la tabla cliente)

Clave Foránea (FK2): vendedor_id (Apunta a la tabla vendedor)

Tabla: detalle_venta

Clave Primaria (PK): Compuesta por (venta_id, producto_codigo)

Clave Foránea (FK1): venta_id (Apunta a la tabla venta)

Clave Foránea (FK2): producto_codigo (Apunta a la tabla producto)

Paso 4. Aplicacion de 3FN

Elimina dependencias transitivas.

Se identificó que en la tabla producto, la columna categoria no depende directamente de la clave primaria (producto_codigo), sino que depende del producto en sí mismo. Para resolver esta dependencia transitiva, se crea una nueva tabla independiente para las categorías.

Separa categorías de productos si la categoria depende del producto y no de la venta.

Se extrae el atributo categoría para formar una nueva entidad de catálogo:

Tabla: categorias

Clave Primaria (PK): categoria_id (Código numérico autoincremental)

Atributos: nombre

Tabla: productos (Modificada)

Clave Primaria (PK): producto_codigo

Clave Foránea (FK): categoria_id (Apunta a la tabla categorias)

Atributos: nombre

Separa ciudades o zonas solo si justificas que se administraran como catalogos.

ciudades (en la tabla cliente): Se decide no separarla en un catálogo independiente porque el volumen de datos actual es muy pequeño y funciona simplemente como un atributo descriptivo de la dirección del cliente.

zonas (en la tabla vendedor): Se mantiene dentro de la tabla vendedor porque cada vendedor tiene asignada una única zona de operación fija y no se requiere por ahora una administración compleja de regiones.

Revisa que cada atributo dependa de la clave, de toda la clave y de nada mas que la clave.

ada atributo de cada tabla depende única y exclusivamente de su clave primaria:

En clientes, todo depende de cliente_doc.

En vendedores, todo depende de vendedor_id.

En categorias, el nombre depende de categoria_id.

En productos, el nombre y la categoría dependen de producto_codigo.

En ventas, los datos de la transacción dependen de venta_id.

En detalle_ventas, las métricas dependen de la clave compuesta (venta_id, producto_codigo)

Paso 5. Diseño del modelo entidad-relacion

Construye el diagrama ER con entidades, atributos, claves y cardinalidades.

Incluye como minimo: clientes, vendedores, ventas, detalle_venta, productos y categorias.

Puedes agregar metodos_pago o ciudades si tu justificacion lo requiere.

No uses la tabla ventas_crudas como tabla transaccional final

Entidad: categorias

categoria_id (PK) [Int]

nombre [Varchar]

Entidad: productos

producto_codigo (PK) [Varchar]

nombre [Varchar]

categoria_id (FK) [Int]

Entidad: clientes

cliente_doc (PK) [Varchar]

nombre [Varchar]

email [Varchar] (Unique)

telefono [Varchar]

direccion [Varchar]

ciudad [Varchar]

Entidad: vendedores

vendedor_id (PK) [Varchar]

nombre [Varchar]

zona [Varchar]

Entidad: ventas

venta_id (PK) [Varchar]

fecha_venta [Date]

cliente_doc (FK) [Varchar]

vendedor_id (FK) [Varchar]

metodo_pago [Varchar]

entidad_pago [Varchar]

Entidad: detalle_ventas

venta_id (PK, FK) [Varchar]

producto_codigo (PK, FK) [Varchar]

cantidad [Int]

precio_unitario [Numeric]

descuento [Numeric]

Relación: Una categoría tiene muchos productos; un producto pertenece a una sola categoría.

Cardinalidad: 1 en categorías --- 1:N (uno a muchos) en productos.

clientes a ventas

Relación: Un cliente puede realizar muchas ventas; cada venta pertenece a un único cliente.

Cardinalidad: 1 en clientes --- 1:N (uno a muchos) en ventas.

vendedores a ventas

Relación: Un vendedor puede atender muchas ventas; cada venta es atendida por un único vendedor.

Cardinalidad: 1 en vendedores --- 1:N (uno a muchos) en ventas.

ventas a detalle_ventas

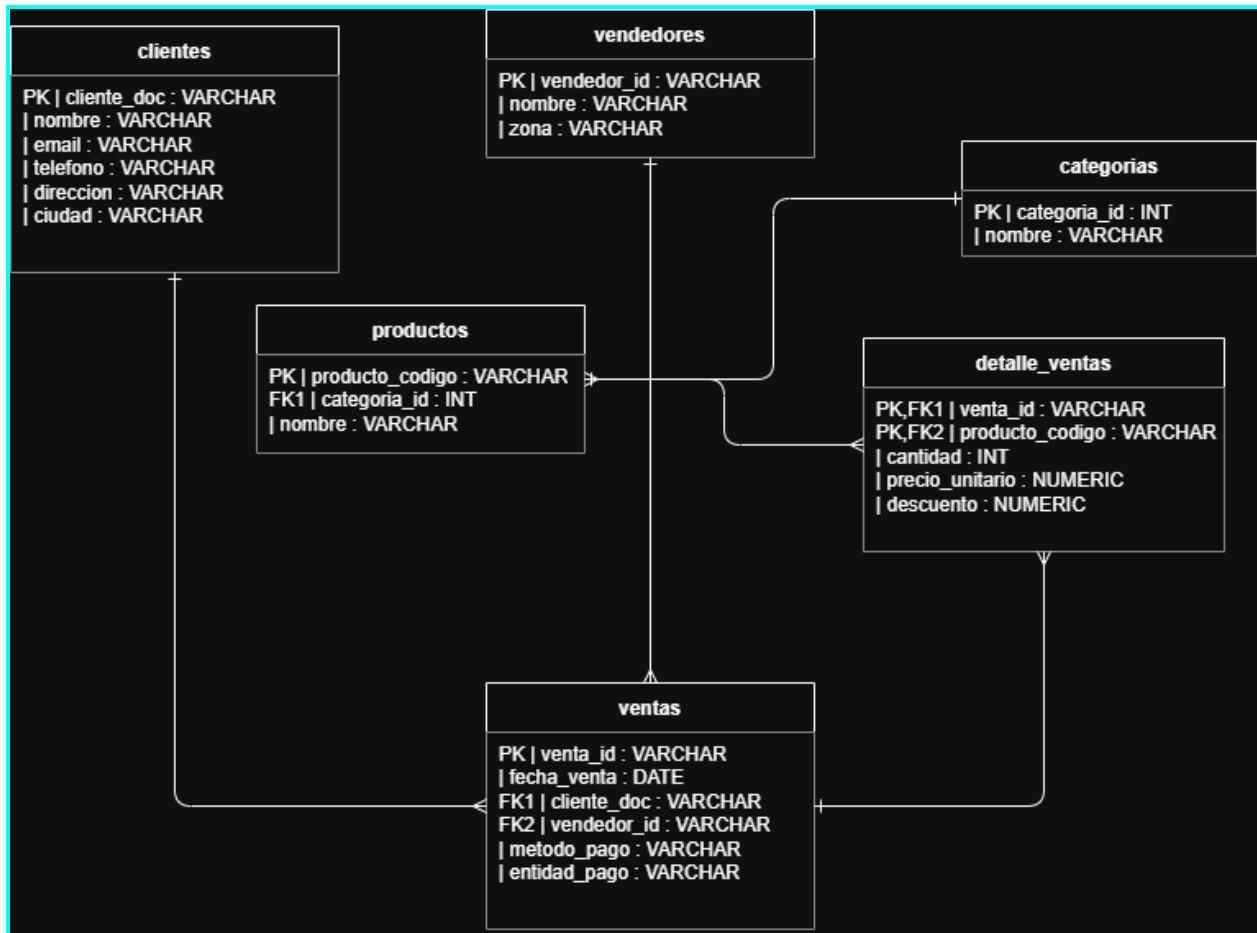
Relación: Una venta puede tener una o muchas líneas de detalle.

Cardinalidad: 1 en ventas --- 1:N (uno a muchos) en detalle_ventas.

productos a detalle_ventas

Relación: Un producto puede aparecer en múltiples líneas de detalle de diferentes ventas.

Cardinalidad: 1 en productos --- 1:N (uno a muchos) en detalle_ventas.



7. Plantilla de analisis de dependencias funcionales

Completa esta tabla en tu informe antes de construir el modelo final

Dependencia funcional	Tipo	Acción de normalización
cliente_doc -> cliente_nombre, cliente_email, cliente_telefono, cliente_direccion	Transitiva (con respecto a la venta) o Completa (de su propio ID)	Separar en una tabla independiente llamada clientes (3FN).

vendedor_id -> vendedor_nombre, vendedor_zona	Transitiva (con respecto a la venta) o Completa (de su propio ID)	Separar en una tabla independiente llamada vendedores (3FN).
producto_codigo -> producto_nombre, categoria	Parcial (con respecto a la clave compuesta) o Completa (de su propio ID)	Separar en una tabla independiente llamada productos (2FN). Posteriormente extraer la categoría a su propia tabla en 3FN.
venta_id -> fecha_venta, cliente_doc, vendedor_id, metodo_pago	Funcional Completa	Mantener como la base o cabecera de la tabla ventas (2FN).
venta_id + producto_codigo -> cantidad, precio_unitario, descuento	Funcional Completa	Crear la tabla intermedia o puente llamada detalle_ventas usando la clave compuesta (2FN).

Consultas

1-total calculado por cada venta desde el detalle

```
SET search_path TO normalizacion_ventas;
```

```
SELECT
    venta_id,
    SUM(cantidad * precio_unitario - descuento) AS total_calculado
FROM detalle_ventas
GROUP BY venta_id
ORDER BY venta_id;
```

Consulta 2: Productos más vendidos por cantidad total

```
SET search_path TO normalizacion_ventas;
```

```
SELECT
    p.producto_codigo,
    p.nombre AS producto_nombre,
    SUM(dv.cantidad) AS unidades_vendidas
FROM detalle_ventas dv
JOIN productos p ON dv.producto_codigo = p.producto_codigo
GROUP BY p.producto_codigo, p.nombre
ORDER BY unidades_vendidas DESC;
```

Consulta 3: Ventas por vendedor

```
SET search_path TO normalizacion_ventas;
```

```
SELECT
    v.vendedor_id,
    v.nombre AS vendedor_nombre,
    COUNT(DISTINCT ve.venta_id) AS cantidad_ventas,
    SUM(dv.cantidad * dv.precio_unitario - dv.descuento) AS
valor_total
FROM vendedores v
LEFT JOIN ventas ve ON v.vendedor_id = ve.vendedor_id
LEFT JOIN detalle_ventas dv ON ve.venta_id = dv.venta_id
GROUP BY v.vendedor_id, v.nombre
ORDER BY valor_total DESC;
```

Consulta 4: Historial de compras de un cliente específico

```
SET search_path TO normalizacion_ventas;
```

```
SELECT
    c.cliente_doc,
    c.nombre AS cliente_nombre,
    v.venta_id,
    v.fecha_venta,
    p.nombre AS producto,
    dv.cantidad
FROM clientes c
JOIN ventas v ON c.cliente_doc = v.cliente_doc
JOIN detalle_ventas dv ON v.venta_id = dv.venta_id
```

```
JOIN productos p ON dv.producto_codigo = p.producto_codigo  
WHERE c.cliente_doc = 'CC101'  
ORDER BY v.fecha_venta, p.nombre;
```

Consulta 5: Control de integridad

```
SET search_path TO normalizacion_ventas;  
  
SELECT COUNT(*) AS registros_huerfanos  
FROM detalle_ventas dv  
LEFT JOIN ventas v ON dv.venta_id = v.venta_id  
LEFT JOIN productos p ON dv.producto_codigo = p.producto_codigo  
WHERE v.venta_id IS NULL OR p.producto_codigo IS NULL;
```